例题 2：应用白盒测试法的具体步骤：

1. 按照某种覆盖标准，选择一组需要覆盖的路径；
2. 确定上述路径所需的输入数据；
3. 计算与输入数据相应的输出。

下面以工资管理程序为例说明白盒测试方法的运用过程。（白盒测试方法有 6 种）

工资管理程序 BONUS 的输入数据是职员表（Employee Table）和部门销售表（Department Table）（图 1）。职员表由姓名（Name）、职务（Job）、部门（Dept.）和工资（Salary）四个属性组成。部门销售表由部门(Dept)和销售量（Sales）组成。程序的功能是："为销售量最大的部门中每一个职工增加 200 元工资，但是，如果某个职员的原有工资已达 15000 元，或者他的职务是经理，则只给他增加 100 元。如果程序能正常地完成，则输出出错码 0；如果两表格中没有任何条目，则输出出错码 1；如果没有职员在部门销售表中销售量最大的部门中工作，则输出出错码 2;"

工资管理程序 BONUS 的两个表如下：

Name	Job	Dept	Salary

Dept	Sales

工资管理程序 BONUS 的源程序：

参数表中 EMPTAB、DEPTTAB 分别是职员表和部门表，ESIZE、DSIZE 分别是两个表的长度，ERRCODE 是出错码。这里略去了说明部分和注释。

```
Procedure     Bonus(EMPTAB,DEPTTAB:TABLE; ESIZE,DSIZE,ERRCODE:INTEGER);
    /*最大销量和出错码初始化*/
1.  Begin MAXSALES:=0; ERRCODE:=0;
    /*判断表是否为空，空输出 1，并退出程序，否则继续执行*/
2.  IF (ESIZE<=0) or (DSIZE<=0)
3.    THEN ERRCODE:=1;
4.  ELSE
    /*获得最大销量*/
5.    For i=1 to DSIZE
6.      If Sales(i)> MAXSALES
7.      then MAXSALES = Sales(i);
    /*找到销量最大的每一个部门，并做进一步处理*/
8.    For j=1 to DSIZE
```

```
If Sales(j)= MAXSALES
Then
    Begin Found:=False;
/*找销量最大的职员所属部门对应的职员表中的职员并处理*/
    For k:=1 to ESIZE
        If (EMPTAB.DEPT(k)= DEPTTAB. DEPT(J))
        Then
            Begin Found:=True;
/*如果该职员工资超过 15000 或职务是经理则加 100 工资，否则加 200 工资*/
            If (SALARY(k)>=15000.0) or (JOB(k)=”M”)
            Then
                SALARY(k):= SALARY(k)+100.0;
            Else
                SALARY(k):= SALARY(k)+200.0;
            End;
/*如果找不到对应职员，则输出出错码 2,否则出错码默认 0*/
```

```
22.         If (!Found ) then ERRORCODE:=2;
23.         End
24.     End
25. End
```

用白盒测试方法设计测试用例。

1.首先列出程序中的判定，考虑所有的条件句和循环句。本例中只要输入表格不空，循环语句总会经历进入循环并且循环次数固定和跳过循环这两种情况（因为循环终值都大于等于循环初值），所以循环我们没有必要专门考虑。需要考虑的只是下面 **6** 个条件语句中的判定。

2. If (ESIZE<=0) or (DSIZE<=0)

6. If Sales(i)> MAXSALES

9. If Sales(j)= MAXSALES

13. If (EMPTAB.DEPT(k)= DEPTTAB. DEPT(J))

16. If (SALARY(k)>=15000.0) or (JOB(k)=”经理”)

22. If (!Found) then ERRORCODE:=2;

2.设计测试用例。

(1)采用“判定覆盖”标准，使得上述 6 个判定都取到两种结果，这就需要使下表列出的 12 种情况都执行到。

判定	结果为“真”	结果为“假”
2	ESIZE 或 DSIZE 等于 0	ESIZE 和 DSIZE 都大于 0
6	总会出现	部门表中，将销售量较小的某个部门放在后面
9	总会出现	部门表中，各部门的销售量不全相等
13	职员表中，有职员在销售量最大的部门工作	职员表中，有职员不在销售量最大的部门工作
16	销售量最大的部门中，有个职员的工资大于等于 15000，或者是“经理”	销售量最大的部门中，有个职员的工资小于 15000，而且不是“经理”
22	销售量最大的部门中没有职员	销售量最大的部门中有职员

下面的测试用例可以满足这一要求，按照原则，每个测试用例要说明输入数据和期望的输出结果（图中“职务”一栏，“E”表示是一般职员，“M”表示经理。）。

例子	输入数据	期望的输出结果
1	ESIZE=0	ERRCODE=1 EMPTAB，DEPTTAB，ESIZE，DSIZE 不变

<table>
<tr>
<td>2</td>
<td>ESIZE= DSIZE=3
EMPTAB DEPTAB
<table>
<tr><td>Name</td><td>Job</td><td>Dept</td><td>Salary</td><td></td><td>Dept</td><td>Sales</td></tr>
<tr><td>Jones</td><td>E</td><td>D42</td><td>16000.0</td><td></td><td>D42</td><td>10000.0</td></tr>
<tr><td>Smith</td><td>E</td><td>D32</td><td>10000.0</td><td></td><td>D32</td><td>8000.0</td></tr>
<tr><td>Lorin</td><td>E</td><td>D42</td><td>14000.0</td><td></td><td>D95</td><td>10000.0</td></tr>
</table></td>
<td>ERRORCODE=2
DEPTTAB，ESIZE，DSIZE 不变
EMPTAB 变为
<table>
<tr><td>Name</td><td>Job</td><td>Dept</td><td>Salary</td></tr>
<tr><td>Jones</td><td>E</td><td>D42</td><td>16100.0</td></tr>
<tr><td>Smith</td><td>E</td><td>D32</td><td>10000.0</td></tr>
<tr><td>Lorin</td><td>E</td><td>D42</td><td>14200.0</td></tr>
</table></td>
</tr>
</table>

虽然这两个例子满足“判定覆盖”标准，但是它们不能发现程序中许多其他可能的错误，例如没有检查 ERRORCODE 为 0、职员是经理、部门表为“空”等情况。

（2）.采用“条件覆盖”标准，即使判定中的每一个条件的两种可能取值都执行到，这就要使下表的 16 中情况出现。

判定	条件	真	假
2	ESIZE<=0	ESIZE<=0	ESIZE>0
2	DSIZE<=0	DSIZE<=0	DSIZE>0
6	Sales(i)> MAXSALES	总会出现	部门表中，将销售量较小的某个部门放在后面
9	Sales(j)= MAXSALES	总会出现	部门表中，各部门的销售量不全相等

13	EMPTAB.DEPT(k)= DEPTTAB. DEPT(J)	职员表中，有职员在销售量最大的部门工作	职员表中，有职员不在销售量最大的部门工作
16	SALARY(k)>=15000.0	销售量最大的部门中，有个职员的工资大于等于 15000	销售量最大的部门中，有个职员的工资小于 15000
16	JOB(k)=”经理”	销售量最大的部门中，有个职员是“经理”	销售量最大的部门中，有个职员不是“经理”
22	!Found	销售量最大的部门中没有职员	销售量最大的部门中有职员

下面的测试用例可以满足这一要求。

<table>
<tr><th>例子</th><th>输入数据</th><th>期望的输出结果</th></tr>
<tr><td>1</td><td>ESIZE=DSIZE=0</td><td>ERRCODE=1
EMPTAB ， DEPTTAB ，ESIZE，DSIZE 不变</td></tr>
<tr><td>2</td><td>ESIZE= DSIZE=3
EMPTAB DEPTAB
<table>
<tr><th>Name</th><th>Job</th><th>Dept</th><th>Salary</th></tr>
<tr><td>Jones</td><td>E</td><td>D42</td><td>16000.0</td></tr>
<tr><td>Smith</td><td>E</td><td>D32</td><td>10000.0</td></tr>
</table>
<table>
<tr><th>Dept</th><th>Sales</th></tr>
<tr><td>D42</td><td>10000.0</td></tr>
<tr><td>D32</td><td>8000.0</td></tr>
</table></td><td>ERRORCODE=2
DEPTTAB，ESIZE，DSIZE 不变
EMPTAB 变为
<table>
<tr><th>Name</th><th>Job</th><th>Dept</th><th>Salary</th></tr>
</table></td></tr>
</table>

	Lorin	M	D42	14000.0		D95	10000.0	Jones	E	D42	16100.0
								Smith	E	D32	10000.0
								Lorin	M	D42	14100.0

虽然这两个例子满足“条件覆盖”标准，但是它们可能比满足“判定覆盖”标准的测试用例还差，因为它们不能执行每一个语句（如语句 19），而且左右也不比“判定覆盖”的多许多，也没有检查 ERRORCODE 为 0，而且如果语句 2 误写成(ESIZE<=0) and (DSIZE<=0)这个错误也不能发现。

（3）采用“判定/条件覆盖”标准，就可克服上面例子的弱点，我们需要提供足够的测试使得所有判定和条件都取到两个不同的值，这里只需要使上面的职员 Jones 为经理，而 Lorin 改成不是经理，则判定 16 就可以取到两种结果，语句 19 因而得到执行。

问题：如果所用的编译系统将含有“or”的表达式处理成：遇到第一项为“真”就不再检查后面的项目，则这样的两个测试用例并不能检查到 JOB(k)=“M”这一部分。

（4）考虑“条件组合覆盖”标准，它需要足够的例子，使得每个判定中条件的各种组合情况都出现一次。本例中判定 6、9、13 和 22 各有两种组合，判定 2 和 16 各有 4 种组合。可以先选出一个测试用例使其包含尽可能多的组合情况。再选另一个测试用例使其包含尽可能多的剩余组合情况…，直到得到一组测试用例能包含所有的组合情况。下面给出满足这一标准的测试用例，而且可以看到它比其他测试方法都全面，这也说明了我们一开始就应该采用这个覆盖标准。

极其重要的一点是：即使是满足“条件组合覆盖”标准的例子仍不能发现所有错误，例如没有检查 ERRORCODE=0 的情况，所以语句 1 被遗漏了就查不出来，又如语句 16 中 15000.0 误写成 15000.01 也是发现不了的，或者 SALARY(k)>=15000.0 误写成 SALARY(k)>15000.0 也是发现不了的；又如果 BONUS 程序没有对部门表或职员表的最后一行进行处理，这个错误也不一定能够发现。

<table>
<tr><th>例子</th><th>输入数据</th><th>期望的输出结果</th></tr>
<tr><td>1</td><td>ESIZE=0　DSIZE=0</td><td>ERRCODE=1
EMPTAB，DEPTTAB，ESIZE，DSIZE 不变</td></tr>
<tr><td>2</td><td>ESIZE=0　DSIZE>0</td><td>同上</td></tr>
<tr><td>3</td><td>ESIZE>0　DSIZE=0</td><td>同上</td></tr>
<tr><td>4</td><td>ESIZE=5　DSIZE=4
EMPTAB
<table><tr><th>Name</th><th>Job</th><th>Dept</th><th>Salary</th></tr><tr><td>Jones</td><td>M</td><td>D42</td><td>16000.0</td></tr><tr><td>Warns</td><td>M</td><td>D95</td><td>12000.0</td></tr><tr><td>Lorin</td><td>E</td><td>D42</td><td>10000.0</td></tr><tr><td>Toy</td><td>E</td><td>D95</td><td>18000.0</td></tr><tr><td>Smith</td><td>E</td><td>D32</td><td>14000.0</td></tr></table>DEPTAB
<table><tr><th>Dept</th><th>Sales</th></tr><tr><td>D42</td><td>10000.0</td></tr><tr><td>D32</td><td>8000.0</td></tr><tr><td>D95</td><td>10000.0</td></tr><tr><td>D44</td><td>10000.0</td></tr></table></td><td>ERRORCODE=2
DEPTTAB，ESIZE，DSIZE 不变
EMPTAB 变为
<table><tr><th>Name</th><th>Job</th><th>Dept</th><th>Salary</th></tr><tr><td>Jones</td><td>M</td><td>D42</td><td>16100.0</td></tr><tr><td>Warns</td><td>M</td><td>D95</td><td>12100.0</td></tr><tr><td>Lorin</td><td>E</td><td>D42</td><td>10200.0</td></tr><tr><td>Toy</td><td>E</td><td>D95</td><td>18100.0</td></tr><tr><td>Smith</td><td>E</td><td>D32</td><td>14000.0</td></tr></table></td></tr>
</table>

通过对工资管理程序的讨论，可以得到两点结论：

“条件组合覆盖”标准比其他标准优越。

即使达到任何一种覆盖标准，其测试效果仍然是不彻底的，我们还需要用其他的测试方法（如黑盒法）作补充。

划分等价类

考虑输入区域

- 加数
  - 数值
    - 整数
      - 正数（1）
      - 0（2）
      - 负数（3）
    - 实数
      - 正实数（4）
      - 负实数（5）

其中，有效的等价类包括：1，2，3，4，5，10。无效的等价类包括：6，7，8，9。

设计测试用例，一个测试用例尽可能包括多个有效的测试用例。一个测试用例只能包括唯一一个无效的测试用例。

然后再考虑包括键盘和鼠标两种输入。所有用例用键盘输入一遍，然后再用鼠标输入一遍。

根据这些设计测试用例如下：

用例编号	覆盖等价类	操作步骤	输入数据	预期的输出
1	1、2	通过鼠标输入数据，点击“=”	“1”、“+”、“0”	1
2	3、4	通过鼠标输入数据，点击“=”	“-1”、“+”、“0.1”	-0.9
3	5、10	通过鼠标输入数据，点击“=”	“-0.1”、“+”、“.”	-0.1
4	7	通过鼠标输入数据，点击“=”	“/”、“+”、“*”	0
5	1、2	通过键盘输入数	“1”、“+”、“0”	1

		据，点击“=”		
6	3、4	通过键盘输入数据，点击“=”	“-1”、“+”、“0.1”	-0.9
7	5、10	通过键盘输入数据，点击“=”	“-0.1”、“+”、“.”	-0.1
8	6	通过键盘输入数据，点击“=”	“a”、“+”、“b”	0
9	7	通过键盘输入数据，点击“=”	“/”、“+”、“*”	0
10	8	通过键盘输入数据，点击“=”	“ ”、“+”、“ ”	0
11	9	通过键盘输入数据，点击“=”	“+”	0